# Lezione 6: Tipi container: list e tuple

Today

## Variabili singole vs contenitori 🔁

- **Concetto chiave**: una variabile normale è un **nome** che contiene **un singolo valore**; un **contenitore** è un **unico nome** che corrisponde a **molti valori**.
  - Esempio pratico: quattro variabili separate (es. temperatura0, temperatura1, temperatura2, temperatura3) sono quattro valori distinti; una lista che contiene quattro temperature è un unico oggetto con quattro valori interni.
  - Differenza fondamentale: un unico nome che indica molti valori rende possibili operazioni (iterazione, slicing, passaggio di collezioni) che con più variabili distinte sarebbero macchinose o impossibili.

## Tipi di contenitori (panoramica) 📦

| Contenitore | Definizione |
|---|---|
| **list** | Sequenza ordinata di valori, sintassi con parentesi quadre `[...]`. |
| **tuple** | Sequenza ordinata e **immutabile**, sintassi con parentesi tonde `(...)`. |
| **set** | Collezione non ordinata di elementi unici. |
| **dictionary** | Mappa chiave→valore (non sequenziale), sintassi con parentesi graffe `{chiave: valore}`. |

## list 📋 — sintassi e caratteristiche

- **Sintassi**: assegnazione con parentesi quadre: nome_variabile = [valore0, valore1, ..., valore$n-1$].

- **Ordinata**: la lista conserva l'ordine di inserimento (non è necessariamente ordine alfabetico, è l'ordine in cui compaiono gli elementi).
- **Tipi misti**: una lista può contenere **int**, **float**, **string**, altre **liste** ecc.; non è necessario che tutti gli elementi siano dello stesso tipo.
- **Esempi utili**: una variabile `settimana = ['lunedì', 'martedì', 'mercoledì', ...]` contiene sette stringhe sotto un unico nome.

## Accesso agli elementi e slicing ✂️

- Indicizzazione: `nome_lista[\(i\)]` restituisce l'elemento in posizione $i$ (le liste sono numerate da $0$).
  - Esempio: `temperature[\(0\)]` → primo valore.
- Slicing: `nome_lista[\(start:stop\)]` funziona come per le stringhe e restituisce una sottosequenza.
- Funzioni utili: $\text{len}(\text{nome_lista})$ restituisce il numero di elementi (es. $\text{len}(\text{temperature}) = 8$ se contiene otto valori).
- Iterazione: `for elemento in nome_lista:` permette di controllare o usare ciascun valore (es. per stampare i tipi con `type(elemento)` ).

## Liste annidate (liste di liste)

- È possibile avere elementi che sono a loro volta liste: `lista = [[a,b], [c,d], ...]` .
- Indicizzazione a più livelli: per ottenere il primo elemento del terzo sotto-lista si usa `lista[\(2\)][\(0\)]` (prima si indica l'elemento della lista principale, poi l'indice nella sottolista).
- Più livelli di annidamento richiedono più parentesi: `lista[\(i\)][\(j\)][\(k\)]` per andare più in profondità.
- Per diagnosi di errori: l'errore comune `List index out of range` significa che l'indice richiesto non esiste nella lista selezionata.

## Esempi pratici e comportamento interattivo 🖥️

- Tipo della struttura: chiedere il tipo con `type(nome_variabile)` restituisce `list` se si tratta di una lista; per un elemento interno `type(nome_lista[\(i\)])` mostra il tipo dell'elemento (es. `int`, `float`, `str`, `list`).
- Liste miste e iterazione mostrano che Python permette elementi di tipo diverso senza definizioni di tipo esplicite (a differenza di C/Java).

## tuple (tupla) 🔒 — sintassi e differenza chiave

- **Sintassi**: `nome_variabile = (valore0, valore1, ..., valore\(n-1\))` con parentesi tonde.
- **Somiglianza con le liste**: la tupla è una sequenza ordinata; accesso con `nome_tupla[\(i\)]` e slicing funzionano come per le liste.
- **Differenza fondamentale**: la **tupla è immutabile** — non è possibile assegnare a singoli elementi (`nome_tupla[\(i\)] = x` produce un errore). È possibile però sostituire l'intera tupla con una nuova assegnazione.
  - Conseguenza pratica: per strutture che non devono cambiare è preferibile usare tuple (garantisce immutabilità degli elementi).

## Ordinamento e funzioni correlate 🔃

- Ordinare liste: usare `sorted(nome_lista)` o `sorted(nome_lista, reverse=True)` per ottenere una versione ordinata (la funzione restituisce una nuova lista ordinata).
- Metodo di lista: esiste anche `nome_lista.sort()` che ordina in-place la lista (modifica l'oggetto esistente).
- Nota: tuple non possono essere ordinate in-place perché immutabili, ma si può usare `sorted(tuple)` per ottenere una lista ordinata a partire da una tupla.

# Debugging per strutture annidate 🛠️

- Bilanciamento parentesi: per creare strutture annidate contare le parentesi aperte e chiuse aiuta a verificare la correttezza.
- Strategia per risolvere richieste di indicizzazione: risolvere da sinistra a destra — ogni indice individua un elemento nella struttura corrente, poi si continua con l'indice successivo sulla struttura risultante fino ad arrivare al valore finale.

# Punti pratici rapidi ✅

- **Indice iniziale**: $0$.
- **len**: $\text{len}(\text{nome_contenitore})$ → numero di elementi.
- **Accesso annidato**: `contenitore[\(i\)][\(j\)]...` per discendere in strutture annidate.
- **Mutabilità**: **list** = mutabile; **tuple** = immutabile.
- **Contenuti**: liste possono contenere qualsiasi tipo, incluse altre liste (liste miste e annidate). 🌐👥📊

# Tupla vs Lista 🐍

## Mutabilità e assegnazione

- **Lista**: è **mutabile** — è possibile assegnare un nuovo valore a un singolo elemento con la sintassi `v[i] = nuovo_valore`. Questo permette operazioni in-place come lo scambio di elementi.
- **Tupla**: è **immutabile** — **non** supporta l'assegnazione ai singoli elementi; si può solo rimpiazzare l'intera tupla con una nuova tupla. Questa è la differenza fondamentale con conseguenze pratiche su operazioni che richiedono modifiche degli elementi.

## Ordinamento: `sorted` e `reverse`

- `sorted(nome_lista, reverse=True)` restituisce una **lista** ordinata in ordine inverso. Il parametro è `reverse=True` (non `reversed`).
- Se passi una **tupla** a `sorted`, il risultato è comunque **una lista**: Python effettua internamente la conversione in lista e restituisce una lista ordinata. Quindi ordinare una tupla non produce una tupla ordinata, ma una nuova lista.
- Nota implicita: operazioni in-place come `list.sort()` modificano la lista esistente, cosa impossibile su tupla (perché richiederebbe assegnazioni agli elementi).

## Conversione tra tipi e modifica temporanea

- Puoi convertire una tupla in lista con `list(tupla)`, modificare elementi sulla lista risultante (es. assegnare `lista[2] = 'Pluto'`) e poi riconvertire in tupla con `tuple(lista)` se necessario. Questo è il modo per "cambiare" un elemento di una tupla senza violare l'immutabilità originaria.

## Perché usare una tupla invece di una lista? 💾

- **Risparmio di memoria**: una tupla occupa meno memoria rispetto a una lista con gli stessi elementi, specialmente su grandi quantità di dati.
- **Velocità**: l'accesso e alcune operazioni possono essere più veloci sulle tuple per via della loro struttura immutabile.
- Regola pratica: usare tuple quando la sequenza è destinata a essere **sola lettura**; usare liste quando servono modifiche frequenti o funzionalità aggiuntive.

## Regole sintattiche e differenze pratiche

| Tipo | Sintassi di inizializzazione |
|---|---|
| **Lista** | variabile = [valore1, valore2, ...] |
| **Tupla** | variabile = (valore1, valore2, ...) |

- **Lettura**: accesso agli elementi con `variabile[i]` in entrambi i casi.
- **Scrittura (assegnazione a elemento)**: possibile solo per **lista** ( `variabile[i] = nuovo_valore` ); non possibile per **tupla**.
- **Operazioni come** `sorted` : su lista restituiscono lista; su tupla restituiscono comunque una lista ordinata (non una tupla).

### Note pratiche ed esami

- Se in un esercizio d'esame viene richiesto esplicitamente di creare una **tupla**, bisogna creare una tupla e non una lista.
- Per la maggior parte degli esercizi didattici potete usare liste se non avete vincoli di memoria, ma ricordate la differenza di comportamento/risultato per funzioni che trasformano il tipo (es. `sorted` ).

## Simulazioni d'esame e prenotazioni 📚

- Le **simulazioni** sono prove pratiche che riproducono il giorno dell'esame (stesso formato operativo): ciascuno al proprio PC in laboratorio, con un compito che comparirà in una cartella condivisa al via.
- Prenotazione obbligatoria tramite **Infostud**: quando compaiono le date per gli esami bisogna prenotarsi sulle sessioni; la stessa procedura vale per le simulazioni. Controllare di essere prenotati.
- Procedura durante la simulazione:
  - Al via si apre la cartella con l'esercizio.
  - Si usa l'IDE preferito (es. Spyder, VS Code) per implementare la soluzione.
  - Si consegna il file; lo staff copia le soluzioni dai PC e procede alla correzione.
- Bonus e calendario:
  - Ogni simulazione dà un piccolo bonus (valore esemplificativo: fino a $0,5$ punti) da aggiungere al voto finale se si supera la simulazione.

a $0,5$ punti) da aggiungere al voto finale se si supera la sir
    - Nel corso del semestre ci saranno più simulazioni (es. 4 pr
    una quinta che è una simulazione completa/vero esame).
    - La quinta simulazione può restituire un voto in trentesimi c
    studente può decidere di usare come voto d'esame reale
    soddisfatto.

- Comunicazioni e dettagli operativi saranno forniti prima delle

    - Nel corso del semestre ci saranno più simulazioni (es. 4 prove più una quinta che è una simulazione completa/vero esame).
    - La quinta simulazione può restituire un voto in trentesimi che lo studente può decidere di usare come voto d'esame reale se soddisfatto.

- Comunicazioni e dettagli operativi saranno forniti prima delle simulazioni (es. istruzioni su come accedere alla cartella e modalità di consegna).

## Logistica e pausa ⏱️

- Durante la lezione è prevista una pausa di 10 minuti (esempio riportato fino alle $11:20$), dopo la quale verrà trattato un argomento ritenuto fondamentale per poter svolgere la simulazione.